



Furkan Büyüksarıkulak

ID: 22002097

EE-102, Section 003

Term Project

Cocktail Machine

31.12.2023

YOUTUBE:

https://youtu.be/i6clyF_ohPQ

AIM:

This project aimed to design a cocktail machine that serve different combinations of 3 drinks using the Basys3 FPGA board.

METHODOLOGY:

First, we created the frame consisting of wood and MDF on which we will build our project and painted it. Water discharge channel, screen brightness adjustment, screen and buttons, power switch and socket are the details added to make the design usable and realistic. The VHDL part of the project aimed to fulfill the following tasks:

- 1) It will drive 12V DC pumps with the L298N driver card corresponding to the situations for 7 different drink combinations, and the operating times of the pumps have been experimentally adjusted according to the filling time of the glass.
- 2) Printing the desired text on the upper and lower lines of a 16x2 LCD screen by using 128 bit binary 2 data.
- 3) Using the right and left buttons, making a selection on created beverage menu via 16x2 screen and it activates the motor drive state corresponding to the preference on the screen as soon as the select button is pressed.

Although the L298N motor driver used has the ability to control the speeds of the motors using the PWM signal, but since such a need was not needed in the design, "in" pins were used for 12V full capacity instead of the "enable" pins used for speed control.

DESIGN SPECIFICATIONS:

Our cocktail machine design is designed in a modular fashion, namely, it consists of 1 top module and 3 sub-modules. The hierarchical design can be observed in (Figure 1).

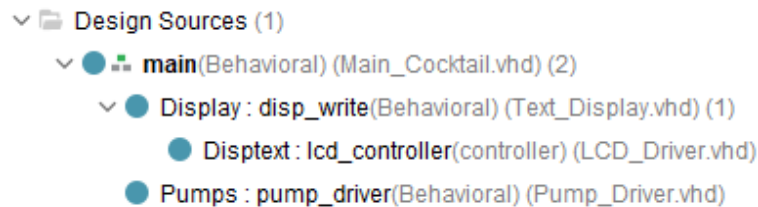


Figure 1 : Hierarchical design

The pump driver module (Pump Driver.vhd) uses a finite state machine (FSM) to determine the behavior of motors in 8 different states, which consist 7 pump state and 1 initial state. The design includes 9 inputs (clk, rst, S_A, S_B, S_C, S_D, S_E, S_F, S_G) and 9 outputs (p1_pin1, p1_pin2, p2_pin1, p2_pin2, p3_pin1, p3_pin2, pump_out_1, pump_out_2, pump_out_3). Inputs such as S_A, S_B represent the operating states of the pumps and are defined when the "current_state" state is "000" and are used to activate the desired state. Outputs such as p1_pin1, p1_pin2 represent the operating logic of the motor driver (L298N) that operates the pumps. There are 2 pins for each pump, for example p3_pin1 is the 1st pin of the 3rd pump. If pins 1 and 2 get the value "1|0", the motor starts to work, and if it gets the value "0|0", the pumps stop working. In the design created by following this logic, it is designed to keep the motors running in the desired state for a period of time to fill the cup, by using a counter, and then to stop the motors by resetting the "current_state" to the initial state "000". Outputs such as pump_out_1 were placed only to experimentally observe the status of the pumps through the LEDs on Basys3. Although it seems pointless, the reason why it is included in the final version of the restriction file will help to observe that this is a problem caused by the pumps in case of a malfunction that may occur in the future. The situation where 3 drinks are mixed and all pumps work for 3 seconds, which is the experimental time required to fill the glass, and then return to the initial state, is as shown in (Figure 2). Also, the RTL scheme of the pump driver module can be observed in (Figure 3).

```

when "111" => --all pump 3 sec
    pump_out_1 <= '1';
    pump_out_2 <= '1';
    pump_out_3 <= '1';
    p1_pin1 <= '1';
    p1_pin2 <= '0';
    p2_pin1 <= '1';
    p2_pin2 <= '0';
    p3_pin1 <= '1';
    p3_pin2 <= '0';
    counter <= counter + 1;
    if counter = (100000000 * 3) then
        current_state <= "000";
    end if;

```

Figure 2 : current_state is "111"

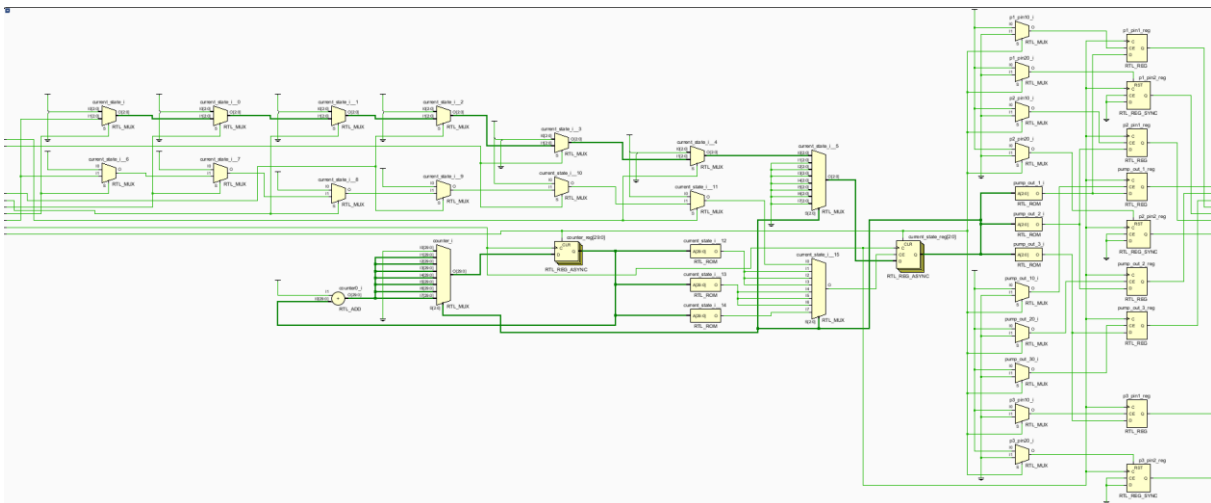


Figure 3 : RTL Schematic of pump driver

The lcd controller module (LCD_Driver.vhd) was created to enable the operation of a 16x2 LCD screen. The screen has 16 pins, the first 2 of which are VSS and VDD, followed by the V0 pin, which is the pin where we can control the brightness of the screen via the potentiometer. The next 3 pins (rs, rw, and e) of which "rs" is the register selection pin, that is, it allows us to choose whether data will be sent to the screen or instructions will be given. The "rw" pin briefly enables selection between writing and reading. If data is to be written to the screen, a "0" signal is required. In addition, since no data is read from the screen in the project, a reading signal "1" is not sent to the "rw" pin in the design. The next "e" pin means

enable, that is, it carries the information about when the data sent from the FPGA will be read by the LCD. There are 8 data pins next to the enable pin. (d0,d1,d2...). These pins allow the actual data/commands to be sent to the screen. Finally, the following anode and cathode pins represent the power and ground of the rear LEDs of the screen. When the module design is examined, there are 4 inputs and 4 outputs, the “entity” part of the module can be seen in (Figure 4). The line1_buffer and line_2 buffer seen carry the information about the characters that will appear in the 16 columns to be sent to the bottom and top lines, and the other inputs and outputs represent the mentioned pins on the LCD. Our module contains a finite state machine with 4 states (power_up, initialize, resetline, send). The “power_up” state represents waiting for the VDD to reach the desired state. “initialize” carries information about the function sets that the screen must perform, while in this state rs is 0. These functions are as follows:

- *lcd_data* <= "00111100"; -- 2-line mode, display on
- *lcd_data* <= "00001100"; -- display on, cursor off, blink off
- *lcd_data* <= "00000001"; -- display clear
- *lcd_data* <= "00000110"; -- increment mode, entire shift off

Then, in the "resetline" state, the 128-bit data for the upper and lower lines received from the user is prepared before being sent, in this case rs is 1. In the "send" state, the requested information is sent to the LCD screen and displayed. If the reset_n button receives the active low signal, the screen returns to the power_up state and the screen is refreshed and becomes ready to receive a new signal from the beginning. The scheme of the design is as shown in the (Figure 5).

```

ENTITY lcd_controller IS
  PORT (
    clk      : IN    STD_LOGIC;  --system clock
    reset_n  : IN    STD_LOGIC;  --active low reinitializes lcd
    rw, rs, e : OUT   STD_LOGIC;  --read/write, setup/data, and enable for lcd
    lcd_data  : OUT   STD_LOGIC_VECTOR(7 DOWNTO 0); --data signals for lcd
    line1_buffer : IN  STD_LOGIC_VECTOR(127 DOWNTO 0); -- Data for the top line of the LCD
    line2_buffer : IN  STD_LOGIC_VECTOR(127 DOWNTO 0); -- Data for the bottom line of the LCD
  )
END lcd_controller;

```

Figure 4 : Entity part of “lcd_controller.vhd”

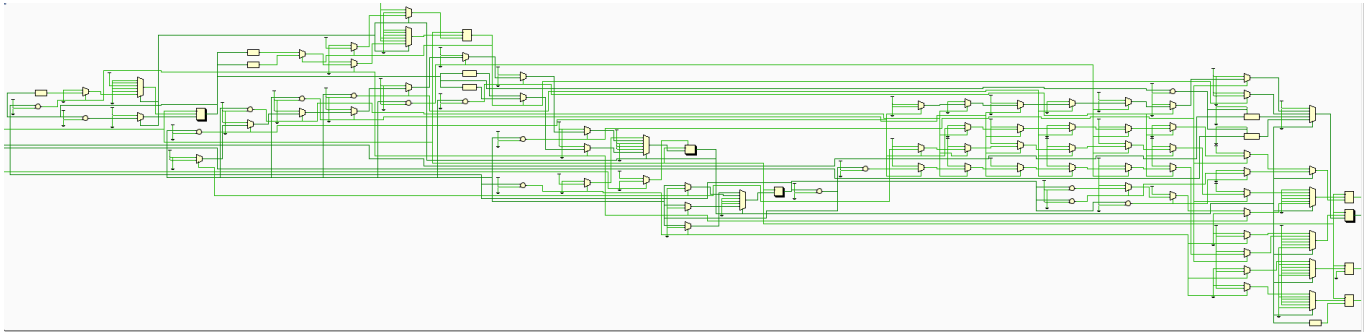


Figure 5 : Schematic of "LCD_Driver.vhd" module

The display write module (Text_Display.vhd) generally controls the data to be written to the screen and the behavior of 3 buttons. The inputs and outputs included in the module are as shown in the (Figure 6). Using the LCD_Driver module we mentioned as a component, the 128-bit data signal of the lines on the screen was created using a text-to-hexadecimal converter. Our process was divided into 2 parts depending on whether the select button was 0 or 1. In the case where the select button is 0, information that will switch to the previous and next menu state has been introduced when the right and left buttons are pressed among the 8 states assigned for the welcome screen and 7 beverage options. With each button pass, a 128-bit binary hexadecimal number was sent to the bottom and top, and the screen was refreshed with the rst button. Here are 2 examples of "text_state" containing hexadecimal information for the welcome and menu screen:

```

case text_state is
  when "000" =>
    top_line  <= x"20202020434F434B5441494C20202020"; -- COCKTAIL
    bottom_line <= x"204D414348494E452062792042534B20"; -- MACHINE by BSK
    .
    .
    .
  when "111" =>
    top_line  <= x"20204D697820436F636B7461696C2020"; -- Mix Cocktail
    bottom_line <= x"202020202053454C4543542020202020"; -- SELECT
end case;

```

Then, the situations in which the selection key was pressed “1” were examined. The last text that appears on the screen when the button is pressed determines the output state, and we will later use this "start_state_out" signal to control the states in the main module. In addition, "Preparing Please Wait..." has been arranged on the screen for the same amount of time that the pump driver remains active in the drink case corresponding to the selected situation. As soon as the drink is filled, the screen becomes ready to enter the menu again by pressing the right or left button on the welcome screen. The schematic of the module is as shown in (Figure 7).

```
entity disp_write is
  Port ( clk : in std_logic;
        lcd_e      : out std_logic;
        lcd_rs     : out std_logic;
        lcd_rw     : out std_logic;
        lcd_db     : out std_logic_vector(7 downto 0);
        btn_right  : in std_logic;
        btn_select : in std_logic;
        btn_left   : in std_logic;
        start_state_out : out std_logic_vector(2 downto 0)
        );
end disp_write;
```

Figure 6 : Entity part of “Text_Display.vhd”

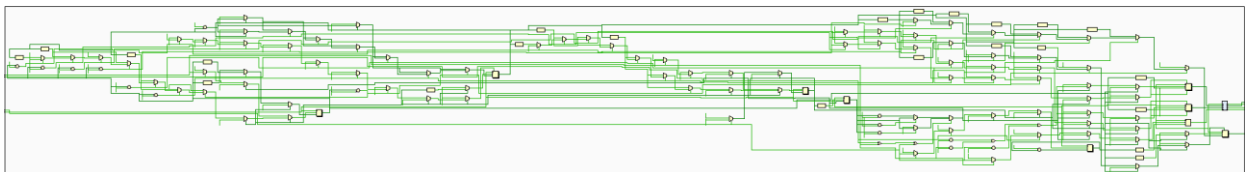


Figure 7 : Schematic of “Text_Display.vhd” module

We used the pump driver and display write module as components in our main module. Using the "start_state_out" output signal that we obtained by activating it with the select button on our display write module, we activated the desired pump states in the pump module and ensured the coordinated operation of the display and pumps. As a result, the person who makes a selection on the screen can enjoy his drink after his glass is filled. The scheme of the design is as shown in (Figure 8).

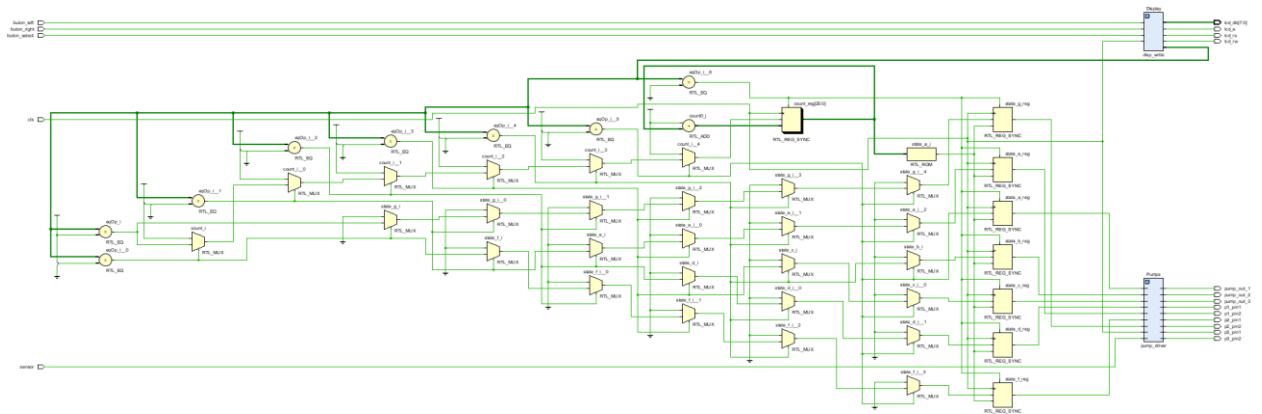


Figure 8 : Schematic of “Main_Cocktail.vhd” top module

Additionally, some images and explanations of the design are as follows:

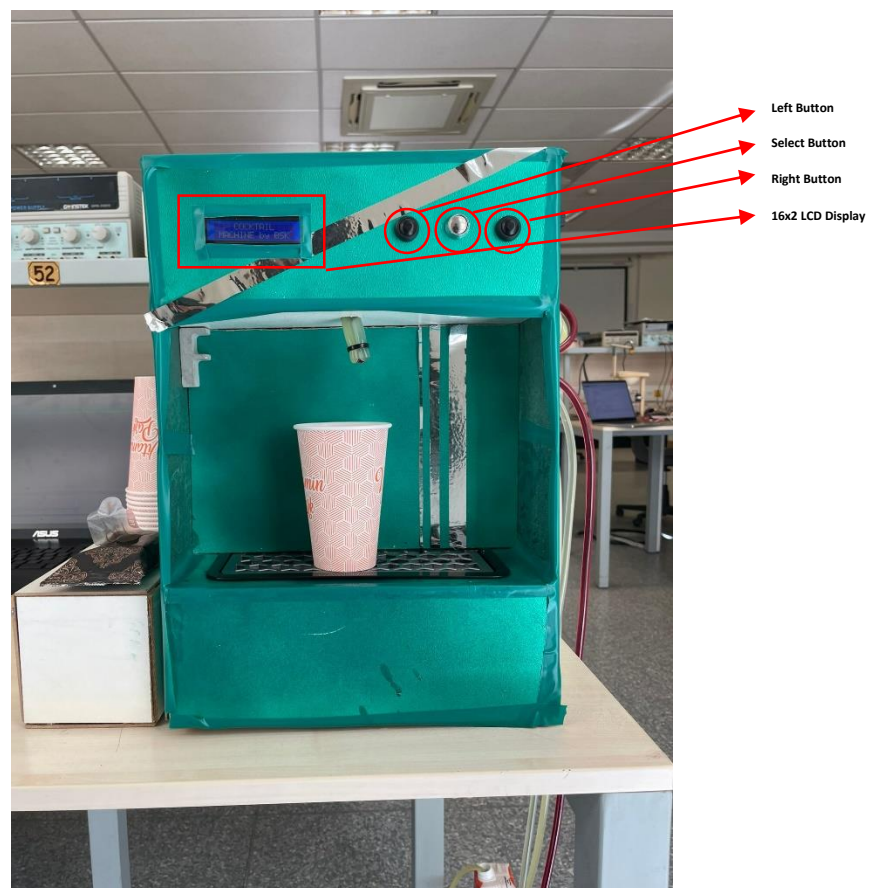


Figure 9 : Cocktail Machine

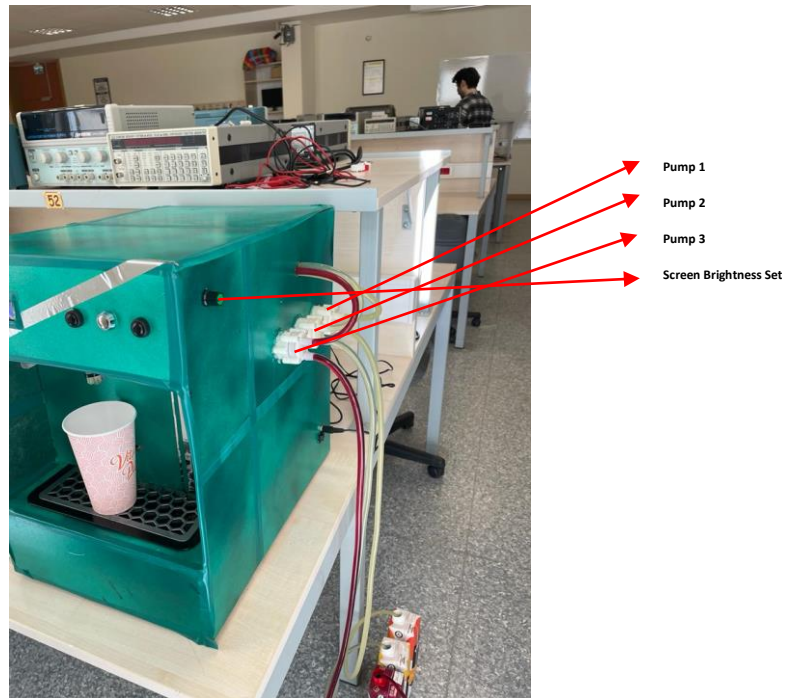


Figure 10 : Pumps and Brightnes Set

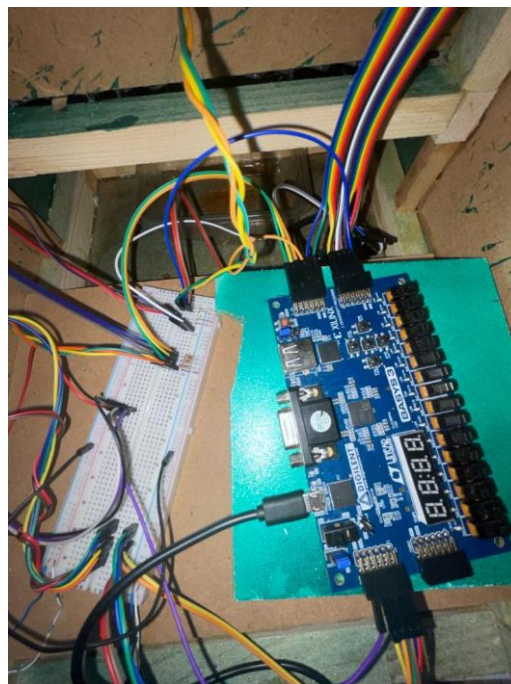


Figure 11 : Basys3 Design Cable Connection

RESULTS:

This machine can offer 7 different drinks in combinations of 3 drinks. The images of the 7 beverage options I mentioned on the menu screen and an example active status are as follows:

CASE1

	Select Buton	
<u>Text State</u>	0	001
<u>Pump State</u>	1	S_A
<u>P1P2P3</u>	1	100

P1 => Pump 1, P2 => Pump 2, P3 => Pump3



Figure 12 : Case 1

CASE2

	Select Buton	
<u>Text State</u>	0	010
<u>Pump State</u>	1	S_B
<u>P1P2P3</u>	1	010

Figure 13 : Case 2



CASE3

	Select Buton	
<u>Text State</u>	0	011
<u>Pump State</u>	1	S_C
P1P2P3	1	001

P1 => Pump 1, P2 => Pump 2, P3 => Pump3



Figure 14 : Case 3

CASE4

	Select Buton	
<u>Text State</u>	0	100
<u>Pump State</u>	1	S_D
P1P2P3	1	110

P1 => Pump 1, P2 => Pump 2, P3 => Pump3



Figure 15 : Case 4

CASE5

	Select Buton	
<u>Text State</u>	0	101
<u>Pump State</u>	1	S_E
P1P2P3	1	101

P1 => Pump 1, P2 => Pump 2, P3 => Pump3



Figure 16 : Case 5

CASE6

	Select Buton	
<u>Text State</u>	0	110
<u>Pump State</u>	1	S_F
P1P2P3	1	011

P1 => Pump 1, P2 => Pump 2, P3 => Pump3



Figure 17 : Case 6

CASE7

	Select Buton	
<u>Text State</u>	0	111
<u>Pump State</u>	1	S_G
P1P2P3	1	111

P1 => Pump 1, P2 => Pump 2, P3 => Pump3



Figure 18 : Case 7



Figure 19 : Example of active state

CONCLUSION:

In this project, I aimed to make a cocktail machine. The purpose of choosing this project was that the product I created would be a functional and enjoyable product that could be used at home even after the project presentation was completed. Based on this idea, I added details such as LCD screen, buttons, adapter, port and switch to the exterior design that I created with wooden materials to make it durable. Thanks to these details, I created a beverage machine that can fulfill its purpose when placed at home. Another project that gave me a lot of benefits during the construction process. I learned how the L298N motor driver works and how it is controlled. Also, even though I quoted the basic code, I learned a lot about the signal processes and communication protocols of LCD displays. In general, I gained experience on how we can implement a VHDL code that can perform a complex task with Basys3 and learned the research process of this. Throughout the project, I encountered numerous errors, problems, and setbacks, both in physical design and software design. I solved the problems I encountered through research and trial and error methods. As a result, I created a cocktail machine that can be used at home, offers 7 combinations of 3 drinks, and is fully functional with buttons and a screen.

REFERENCES:

[1] M. Panchal, "16x2-LCD-Controller-VHDL," GitHub, 2017.

<https://github.com/Maeur1/16x2-LCD-Controller-VHDL> (accessed Dec. 4, 2023).

APPENDIX:

Main Cocktail.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;

entity main is
Port (
    clk          : in std_logic;
    sensor       : in std_logic;
    p1_pin1      : out std_logic;
    p1_pin2      : out std_logic;
    p2_pin1      : out std_logic;
    p2_pin2      : out std_logic;
    p3_pin1      : out std_logic;
    p3_pin2      : out std_logic;
    pump_out_1   : out std_logic; -----
    pump_out_2   : out std_logic; -- Just to observe pump state on Basys led's
    pump_out_3   : out std_logic; -----
    buton_right  : in std_logic;
    buton_select : in std_logic;
    buton_left   : in std_logic;
    lcd_e        : out std_logic;
    lcd_rs       : out std_logic;
    lcd_rw       : out std_logic;
    lcd_db       : out std_logic_vector(7 downto 0)
);

end main;
```

architecture Behavioral of main is

COMPONENT disp_write IS

```
    PORT(  
        clk      : in std_logic;  
        lcd_e     : out std_logic;  
        lcd_rs    : out std_logic;  
        lcd_rw    : out std_logic;  
        lcd_db    : out std_logic_vector(7 downto 0);  
        btn_right  : in std_logic;  
        btn_select : in std_logic;  
        btn_left   : in std_logic;  
        start_state_out : out std_logic_vector(2 downto 0)  
    );  
END COMPONENT;
```

COMPONENT pump_driver IS

```
Port ( clk : in STD_LOGIC;  
    reset : in STD_LOGIC;  
    p1_pin1 : out STD_LOGIC;  
    p1_pin2 : out STD_LOGIC;  
    p2_pin1 : out STD_LOGIC;  
    p2_pin2 : out STD_LOGIC;  
    p3_pin1 : out STD_LOGIC;  
    p3_pin2 : out STD_LOGIC;  
    pump_out_1 : out STD_LOGIC;  
    pump_out_2 : out STD_LOGIC;  
    pump_out_3 : out STD_LOGIC;  
    S_A : in std_logic;
```



```

        S_B : in std_logic;

        S_C : in std_logic;

        S_D : in std_logic;

        S_E : in std_logic;

        S_F : in std_logic;

        S_G : in std_logic

    );

end COMPONENT;

    signal state_a, state_b, state_c, state_d, state_e, state_f, state_g : std_logic;

    signal start_state_outt : std_logic_vector(2 downto 0) := "000";

    signal count : INTEGER RANGE 0 TO 50000000 := 0;

begin

```

```

Display: disp_write port map(

    clk      => clk,

    lcd_e     => lcd_e,

    lcd_rs    => lcd_rs,

    lcd_rw    => lcd_rw,

    lcd_db    => lcd_db,

    btn_right  => buton_right,

    btn_select => buton_select,

    btn_left   => buton_left,

    start_state_out => start_state_outt

);

```

```

Pumps: pump_driver port map(

    clk      => clk ,

    reset     => sensor ,

    p1_pin1   => p1_pin1 ,

    p1_pin2   => p1_pin2 ,

```

```

p2_pin1    => p2_pin1 ,
p2_pin2    => p2_pin2 ,
p3_pin1    => p3_pin1 ,
p3_pin2    => p3_pin2 ,
pump_out_1 => pump_out_1,
pump_out_2 => pump_out_2,
pump_out_3 => pump_out_3,
S_A        => state_a  ,
S_B        => state_b  ,
S_C        => state_c  ,
S_D        => state_d  ,
S_E        => state_e  ,
S_F        => state_f  ,
S_G        => state_g
);

```

```

process(clk, buton_select)

```

```

begin

```

```

    if rising_edge(clk) then

```

```

        if start_state_outt = "000" then -- Pump State Activation

```

```

            count <= 0;

```

```

            state_a <= '0';

```

```

            state_b <= '0';

```

```

            state_c <= '0';

```

```

            state_d <= '0';

```

```

            state_e <= '0';

```

```

            state_f <= '0';

```

```

            state_g <= '0';

```

```

        elsif start_state_outt = "001" then -- State A (S_A) Activation

```

```

            state_a <= '1';

```

```

            count <= count +1;

```

```

            if count = 50000000 then

```

```

                state_a <= '0';

```

```
end if;

elsif start_state_outt = "010" then -- State B (S_B) Activation
    state_b <= '1';
    count <= count +1;
    if count = 50000000 then
        state_b <= '0';
    end if;
elsif start_state_outt = "011" then -- State C (S_C) Activation
    state_c <= '1';
    count <= count +1;
    if count = 50000000 then
        state_c <= '0';
    end if;
elsif start_state_outt = "100" then -- State D (S_D) Activation
    state_d <= '1';
    count <= count +1;
    if count = 50000000 then
        state_d <= '0';
    end if;
elsif start_state_outt = "101" then -- State E (S_E) Activation
    state_e <= '1';
    count <= count +1;
    if count = 50000000 then
        state_e <= '0';
    end if;
elsif start_state_outt = "110" then -- State F (S_F) Activation
    state_f <= '1';
    count <= count +1;
    if count = 50000000 then
        state_f <= '0';
    end if;
elsif start_state_outt = "111" then -- State G (S_G) Activation
    state_g <= '1';
```

```

        count <= count +1;

        if count = 50000000 then
            state_g <= '0';
        end if;
    end if;
end if;
end process;

```

```

end Behavioral;

```

Pump_Driver.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity pump_driver is
    Port ( clk : in std_logic;
          reset : in std_logic;
          p1_pin1 : out std_logic;
          p1_pin2 : out std_logic;
          p2_pin1 : out std_logic;
          p2_pin2 : out std_logic;
          p3_pin1 : out std_logic;
          p3_pin2 : out std_logic;
          pump_out_1 : out std_logic;
          pump_out_2 : out std_logic;
          pump_out_3 : out std_logic;
          S_A : in std_logic; --only p1 10 sec
          S_B : in std_logic; --only p2 10 sec
          S_C : in std_logic; --only p3 10 sec
    );
end entity;

```

```

    S_D : in std_logic; --p1 and p2 5 sec
    S_E : in std_logic; --p1 and p3 5 sec
    S_F : in std_logic; --p2 and p3 5 sec
    S_G : in std_logic); --all pumps 3 sec
end pump_driver;

```

architecture Behavioral of pump_driver is

```

    signal current_state : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal counter : INTEGER RANGE 0 TO (100000000 * 10) := 0; --100M clock cycles at 100MHz clock
begin

```

```

process (clk, reset, S_A, S_B, S_C, S_D, S_E, S_F, S_G)

```

```

begin

```

```

    if reset = '1' then

```

```

        current_state <= "000";

```

```

        counter <= 0;

```

```

    elsif rising_edge(clk) then

```

```

        case current_state is

```

```

            when "000" => --initial state

```

```

                pump_out_1 <= '0';

```

```

                pump_out_2 <= '0';

```

```

                pump_out_3 <= '0';

```

```

                p1_pin1 <= '0';

```

```

                p1_pin2 <= '0';

```

```

                p2_pin1 <= '0';

```

```

                p2_pin2 <= '0';

```

```

                p3_pin1 <= '0';

```

```

                p3_pin2 <= '0';

```

```

        if S_A = '1' then

```

```

            current_state <= "001";

```

```

elseif S_B = '1' then
    current_state <= "010";
elseif S_C = '1' then
    current_state <= "011";
elseif S_D = '1' then
    current_state <= "100";    --State selection
elseif S_E = '1' then
    current_state <= "101";
elseif S_F = '1' then
    current_state <= "110";
elseif S_G = '1' then
    current_state <= "111";
end if;

```

```

    counter <= 0;
--behaviour of situations
when "001" => --only p1 10 sec
    pump_out_1 <= '1';
    pump_out_2 <= '0';
    pump_out_3 <= '0';
    p1_pin1 <= '1';
    p1_pin2 <= '0';
    p2_pin1 <= '0';
    p2_pin2 <= '0';
    p3_pin1 <= '0';
    p3_pin2 <= '0';
    counter <= counter + 1;
    if counter = (100000000 * 10) then
        current_state <= "000";
    end if;

```

```

when "010" => --only p2 10 sec
    pump_out_1 <= '0';

```

```
pump_out_2 <= '1';
pump_out_3 <= '0';
p1_pin1 <= '0';
p1_pin2 <= '0';
p2_pin1 <= '1';
p2_pin2 <= '0';
p3_pin1 <= '0';
p3_pin2 <= '0';
counter <= counter + 1;
if counter = (100000000 * 10) then
    current_state <= "000";
end if;
```

when "011" => --only p3 10 sec

```
pump_out_1 <= '0';
pump_out_2 <= '0';
pump_out_3 <= '1';
p1_pin1 <= '0';
p1_pin2 <= '0';
p2_pin1 <= '0';
p2_pin2 <= '0';
p3_pin1 <= '1';
p3_pin2 <= '0';
counter <= counter + 1;
if counter = (100000000 * 10) then
    current_state <= "000";
end if;
```

when "100" => --p1 and p2 5 sec

```
pump_out_1 <= '1';
pump_out_2 <= '1';
pump_out_3 <= '0';
p1_pin1 <= '1';
```

```
p1_pin2 <= '0';  
p2_pin1 <= '1';  
p2_pin2 <= '0';  
p3_pin1 <= '0';  
p3_pin2 <= '0';  
counter <= counter + 1;  
if counter = (100000000 * 5) then  
    current_state <= "000";  
end if;
```

when "101" => --p1 and p3 5 sec

```
pump_out_1 <= '1';  
pump_out_2 <= '0';  
pump_out_3 <= '1';  
p1_pin1 <= '1';  
p1_pin2 <= '0';  
p2_pin1 <= '0';  
p2_pin2 <= '0';  
p3_pin1 <= '1';  
p3_pin2 <= '0';  
counter <= counter + 1;  
if counter = (100000000 * 5) then  
    current_state <= "000";  
end if;
```

when "110" => --p2 and p3 5 sec

```
pump_out_1 <= '0';  
pump_out_2 <= '1';  
pump_out_3 <= '1';  
p1_pin1 <= '0';  
p1_pin2 <= '0';  
p2_pin1 <= '1';  
p2_pin2 <= '0';
```



```

    p3_pin1 <= '1';
    p3_pin2 <= '0';
    counter <= counter + 1;
    if counter = (100000000 * 5) then
        current_state <= "000";
    end if;

when "111" => --all pump 3 sec
    pump_out_1 <= '1';
    pump_out_2 <= '1';
    pump_out_3 <= '1';
    p1_pin1 <= '1';
    p1_pin2 <= '0';
    p2_pin1 <= '1';
    p2_pin2 <= '0';
    p3_pin1 <= '1';
    p3_pin2 <= '0';
    counter <= counter + 1;
    if counter = (100000000 * 3) then
        current_state <= "000";
    end if;

-----

    when others =>
        current_state <= "000";
        counter <= 0;
    end case;
end if;
end process;
end Behavioral;

```

LCD_Driver.vhd

LIBRARY ieee;

USE ieee.std_logic_1164.ALL;

ENTITY lcd_controller IS

PORT(

clk : IN STD_LOGIC; --system clock

reset_n : IN STD_LOGIC; --active low reinitializes lcd

rw, rs, e : OUT STD_LOGIC; --read/write, setup/data, and enable for lcd

lcd_data : OUT STD_LOGIC_VECTOR(7 DOWNT0 0); --data signals for lcd

line1_buffer : IN STD_LOGIC_VECTOR(127 downto 0); -- Data for the top line of the LCD

line2_buffer : IN STD_LOGIC_VECTOR(127 downto 0)); -- Data for the bottom line of the LCD

END lcd_controller;

ARCHITECTURE controller OF lcd_controller IS

TYPE CONTROL IS(power_up, initialize, RESETLINE, line1, line2, send);

SIGNAL state : CONTROL;

CONSTANT freq : INTEGER := 100; --system clock frequency in MHz

SIGNAL ptr : natural range 0 to 16 := 15; -- To keep track of what character we are up to

SIGNAL line : STD_LOGIC := '1';

BEGIN

PROCESS(clk)

VARIABLE clk_count : INTEGER := 0; --event counter for timing

BEGIN

IF(clk'EVENT and clk = '1') THEN

CASE state IS

--wait 50 ms to ensure Vdd has risen and required LCD wait is met

WHEN power_up =>

IF(clk_count < (50000 * freq)) THEN --wait 50 ms

clk_count := clk_count + 1;

```

    state <= power_up;
ELSE
    --power-up complete

    clk_count := 0;

    rs <= '0';

    rw <= '0';

    lcd_data <= "00110000";

    state <= initialize;
END IF;

--cycle through initialization sequence
WHEN initialize =>

    clk_count := clk_count + 1;

    IF(clk_count < (10 * freq)) THEN    --function set
        lcd_data <= "00111100";    --2-line mode, display on
        e <= '1';

        state <= initialize;
    ELSIF(clk_count < (60 * freq)) THEN    --wait 50 us
        lcd_data <= "00000000";

        e <= '0';

        state <= initialize;
    ELSIF(clk_count < (70 * freq)) THEN    --display on/off control
        lcd_data <= "00001100";    --display on, cursor off, blink off
        e <= '1';

        state <= initialize;
    ELSIF(clk_count < (120 * freq)) THEN    --wait 50 us
        lcd_data <= "00000000";

        e <= '0';

        state <= initialize;
    ELSIF(clk_count < (130 * freq)) THEN    --display clear
        lcd_data <= "00000001";

        e <= '1';

        state <= initialize;
    ELSIF(clk_count < (2130 * freq)) THEN    --wait 2 ms

```

```

    lcd_data <= "00000000";
    e <= '0';
    state <= initialize;
ELSIF(clk_count < (2140 * freq)) THEN --entry mode set
    lcd_data <= "00000110";    --increment mode, entire shift off
    e <= '1';
    state <= initialize;
ELSIF(clk_count < (2200 * freq)) THEN --wait 60 us
    lcd_data <= "00000000";
    e <= '0';
    state <= initialize;
ELSE
    --initialization complete
    clk_count := 0;
    state <= RESETLINE;
END IF;

```

```

        WHEN resetline =>
            ptr <= 16;
            if line = '1' then
                lcd_data <= "10000000";
                rs <= '0';
                rw <= '0';
                clk_count := 0;
                state <= send;
            else
                lcd_data <= "11000000";
                rs <= '0';
                rw <= '0';
                clk_count := 0;
                state <= send;
            end if;

```

```

WHEN line1 =>

```

```

        line <= '1';
        lcd_data <= line1_buffer(ptr*8 + 7 downto ptr*8);

rs <= '1';
rw <= '0';
clk_count := 0;

        line <= '1';

state <= send;

    WHEN line2 =>
        line <= '0';
        lcd_data <= line2_buffer(ptr*8 + 7 downto ptr*8);

rs <= '1';
rw <= '0';
clk_count := 0;
state <= send;

--send instruction to lcd
WHEN send =>
    IF(clk_count < (50 * freq)) THEN --do not exit for 50us
        IF(clk_count < freq) THEN    --negative enable
            e <= '0';
        ELSIF(clk_count < (14 * freq)) THEN --positive enable half-cycle
            e <= '1';
        ELSIF(clk_count < (27 * freq)) THEN --negative enable half-cycle
            e <= '0';
        END IF;
        clk_count := clk_count + 1;
        state <= send;
    ELSE
        clk_count := 0;
        if line = '1' then
            if ptr = 0 then
                line <= '0';
            end if;
        end if;
    end if;
end if;

```

```

                                state <= resetline;
                        else
                                ptr <= ptr - 1;
                                state <= line1;
                        end if;
                else
                        if ptr = 0 then
                                line <= '1';
                                state <= resetline;
                        else
                                ptr <= ptr - 1;
                                state <= line2;
                        end if;
                end if;
        END IF;

END CASE;

--reset
IF(reset_n = '0') THEN
        state <= power_up;
END IF;

END IF;
END PROCESS;
END controller;

```

Text Display.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.STD_LOGIC_ARITH.ALL;
```

```
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
entity disp_write is
```

```
    Port ( clk : in std_logic;
```

```
           lcd_e    : out std_logic;
```

```
           lcd_rs    : out std_logic;
```

```
           lcd_rw    : out std_logic;
```

```
           lcd_db    : out std_logic_vector(7 downto 0);
```

```
           btn_right : in std_logic;
```

```
           btn_select: in std_logic;
```

```
           btn_left  : in std_logic;
```

```
           start_state_out : out std_logic_vector(2 downto 0)
```

```
    );
```

```
end disp_write;
```

```
architecture Behavioral of disp_write is
```

```
    COMPONENT lcd_controller IS
```

```
        Port(
```

```
            clk      : in  std_logic; --system clock
```

```
            reset_n  : in  std_logic; --active low reinitializes lcd
```

```
            rw, rs, e : out std_logic; --read/write, setup/data, and enable for lcd
```

```
            lcd_data  : out std_logic_vector(7 downto 0); --data signals for lcd
```

```
            line1_buffer : in std_logic_vector(127 downto 0); -- Data for the top line of the LCD
```

```
            line2_buffer : in std_logic_vector(127 downto 0)); -- Data for the bottom line of the
```

```
LCD
```

```
        end component;
```

```
        signal top_line : std_logic_vector(127 downto 0);
```

```

        signal bottom_line : std_logic_vector(127 downto 0);
signal text_state : std_logic_vector(2 downto 0) := "000";
signal start_state : INTEGER := 0;
signal count_disp : INTEGER RANGE 0 TO (100000000 * 10) := 0;
signal count_rst : INTEGER RANGE 0 TO 50000000 := 0;
signal rst : std_logic := '0';

begin

Disptext: lcd_controller port map(
    clk => clk,
    reset_n => rst,
    e => lcd_e,
    rs => lcd_rs,
    rw => lcd_rw,
    lcd_data => lcd_db,
    line1_buffer => top_line,
    line2_buffer => bottom_line
);

process(clk,btn_right,btn_left)
    variable debounce_count : integer:= 0;
    variable debounce_limit : integer:= 50;
begin
    if rising_edge(clk) then
        if start_state = 0 then
            if btn_right = '1' or btn_left = '1' or btn_select = '1' then
                if debounce_count <= debounce_limit then
                    debounce_count := debounce_count +1;
                end if;
            else
                if debounce_count > 0 then

```



```

        debounce_count := 0;
    end if;
end if;
if debounce_count = debounce_limit then
    if btn_right = '1' then-- right button menu transitions
        case text_state is
            when "000" =>
                text_state <= "001";
                rst <= '1';
                count_rst <= count_rst +1;
                if count_rst = 50000000 then
                    rst <= '0';
                end if;
            when "001" =>
                text_state <= "010";
                rst <= '1';
                count_rst <= count_rst +1;
                if count_rst = 50000000 then
                    rst <= '0';
                end if;
            when "010" =>
                text_state <= "011";
                rst <= '1';
                count_rst <= count_rst +1;
                if count_rst = 50000000 then
                    rst <= '0';
                end if;
            when "011" =>
                text_state <= "100";
                rst <= '1';
                count_rst <= count_rst +1;
                if count_rst = 50000000 then
                    rst <= '0';

```

```
    end if;
when "100" =>
    text_state <= "101";
    rst <= '1';
    count_rst <= count_rst +1;
    if count_rst = 50000000 then
        rst <= '0';
    end if;
when "101" =>
    text_state <= "110";
    rst <= '1';
    count_rst <= count_rst +1;
    if count_rst = 50000000 then
        rst <= '0';
    end if;
when "110" =>
    text_state <= "111";
    rst <= '1';
    count_rst <= count_rst +1;
    if count_rst = 50000000 then
        rst <= '0';
    end if;
when "111" =>
    text_state <= "001";
    rst <= '1';
    count_rst <= count_rst +1;
    if count_rst = 50000000 then
        rst <= '0';
    end if;
when others =>
    text_state <= "000";
    rst <= '1';
    count_rst <= count_rst +1;
```

```
    if count_rst = 50000000 then
        rst <= '0';
    end if;
end case;
```

```
elsif btn_left = '1' then --left button menu transitions
```

```
case text_state is
```

```
when "000" =>
```

```
    text_state <= "111";
```

```
    rst <= '1';
```

```
    count_rst <= count_rst +1;
```

```
    if count_rst = 50000000 then
```

```
        rst <= '0';
```

```
    end if;
```

```
when "111" =>
```

```
    text_state <= "110";
```

```
    rst <= '1';
```

```
    count_rst <= count_rst +1;
```

```
    if count_rst = 50000000 then
```

```
        rst <= '0';
```

```
    end if;
```

```
when "110" =>
```

```
    text_state <= "101";
```

```
    rst <= '1';
```

```
    count_rst <= count_rst +1;
```

```
    if count_rst = 50000000 then
```

```
        rst <= '0';
```

```
    end if;
```

```
when "101" =>
```

```
    text_state <= "100";
```

```
    rst <= '1';
```

```
    count_rst <= count_rst +1;
```

```
    if count_rst = 50000000 then
        rst <= '0';
    end if;
when "100" =>
    text_state <= "011";
    rst <= '1';
    count_rst <= count_rst +1;
    if count_rst = 50000000 then
        rst <= '0';
    end if;
when "011" =>
    text_state <= "010";
    rst <= '1';
    count_rst <= count_rst +1;
    if count_rst = 50000000 then
        rst <= '0';
    end if;
when "010" =>
    text_state <= "001";
    rst <= '1';
    count_rst <= count_rst +1;
    if count_rst = 50000000 then
        rst <= '0';
    end if;
when "001" =>
    text_state <= "111";
    rst <= '1';
    count_rst <= count_rst +1;
    if count_rst = 50000000 then
        rst <= '0';
    end if;
when others =>
    text_state <= "000";
```

```

        rst <= '1';

        count_rst <= count_rst +1;

        if count_rst = 50000000 then

            rst <= '0';

        end if;

    end case;

elsif btn_select = '1' then -- confirm / select button action

    start_state <= 1;

    rst <= '1';

    count_rst <= count_rst +1;

        if count_rst = 50000000 then

            rst <= '0';

        end if;

    else start_state <= 0;

    end if;

elsif btn_right = '0' and btn_left = '0' and btn_select = '0' then --beverage menu

    case text_state is

        when "000" =>

            top_line  <= x"20202020434F434B5441494C20202020"; -- COCKTAIL

            bottom_line <= x"204D414348494E452062792042534B20"; -- MACHINE by BSK

        when "001" =>

            top_line  <= x"20204F72616E6765204A756963652020"; -- Orange Juice

            bottom_line <= x"202020202053454C4543542020202020"; -- SELECT

        when "010" =>

            top_line  <= x"50696E656170706C65204A7569636520"; -- Pineapple Juice

            bottom_line <= x"202020202053454C4543542020202020"; -- SELECT

        when "011" =>

            top_line  <= x"2020436865727279204A756963652020"; -- Cherry Juice

            bottom_line <= x"202020202053454C4543542020202020"; -- SELECT

        when "100" =>

            top_line  <= x"4F72616E67652D50696E656170706C65"; -- Orange-Pineapple

            bottom_line <= x"202020202053454C4543542020202020"; -- SELECT

        when "101" =>

```

```

        top_line  <= x"204F72616E67652D4368657272792020"; -- Orange-Cherry
        bottom_line <= x"202020202053454C4543542020202020"; -- SELECT
when "110" =>
        top_line  <= x"50696E656170706C652D436865727279"; -- Pineapple-Cherry
        bottom_line <= x"202020202053454C4543542020202020"; -- SELECT
when "111" =>
        top_line  <= x"20204D697820436F636B7461696C2020"; -- Mix Cocktail
        bottom_line <= x"202020202053454C4543542020202020"; -- SELECT
end case;
end if;
elsif start_state = 1 then -- Beverage preparation screen
if text_state = "001" then
        start_state_out <= "001";--s_a
        top_line <=  x"202020507265706172696E672020202020"; -- Preparing
        bottom_line <= x"20506C6561736520576169742E2E2E20"; -- Please Wait...
        count_disp <= count_disp +1;
        if count_disp = (100000000 * 10) then
                text_state <= "000";
                start_state_out <= "000";
                start_state<= 0;
                count_disp <= 0;
        end if;

elsif text_state = "010" then
        start_state_out <= "010";--s_b
        top_line <=  x"202020507265706172696E672020202020"; -- Preparing
        bottom_line <= x"20506C6561736520576169742E2E2E20"; -- Please Wait...
        count_disp <= count_disp +1;
        if count_disp = (100000000 * 10) then
                text_state <= "000";
                start_state_out <= "000";
                start_state<= 0;
                count_disp <= 0;

```

```

end if;
elsif text_state = "011" then
    start_state_out <= "011";--s_c
    top_line <= x"202020507265706172696E6720202020"; -- Preparing
    bottom_line <= x"20506C6561736520576169742E2E2E20"; -- Please Wait...
    count_disp <= count_disp +1;
    if count_disp = (100000000 * 10) then
        text_state <= "000";
        start_state_out <= "000";
        start_state<= 0;
        count_disp <= 0;
    end if;
elsif text_state = "100" then
    start_state_out <= "100";--s_d
    top_line <= x"202020507265706172696E6720202020"; -- Preparing
    bottom_line <= x"20506C6561736520576169742E2E2E20"; -- Please Wait...
    count_disp <= count_disp +1;
    if count_disp = (100000000 * 5) then
        text_state <= "000";
        start_state_out <= "000";
        start_state<= 0;
        count_disp <= 0;
    end if;
elsif text_state = "101" then
    start_state_out <= "101";--s_f
    top_line <= x"202020507265706172696E6720202020"; -- Preparing
    bottom_line <= x"20506C6561736520576169742E2E2E20"; -- Please Wait...
    count_disp <= count_disp +1;
    if count_disp = (100000000 * 5) then
        text_state <= "000";
        start_state_out <= "000";
        start_state<= 0;
        count_disp <= 0;
    end if;

```

```

    end if;
elseif text_state = "110" then
    start_state_out <= "110";--s_e
    top_line <=  x"202020507265706172696E6720202020"; -- Preparing
    bottom_line <= x"20506C6561736520576169742E2E2E20"; -- Please Wait...
    count_disp <= count_disp +1;
    if count_disp = (100000000 * 5) then
        text_state <= "000";
        start_state_out <= "000";
        start_state<= 0;
        count_disp <= 0;
    end if;
elseif text_state = "111" then
    start_state_out <= "111";--s_g
    top_line <=  x"202020507265706172696E6720202020"; -- Preparing
    bottom_line <= x"20506C6561736520576169742E2E2E20"; -- Please Wait...
    count_disp <= count_disp +1;
    if count_disp = (100000000 * 3) then
        text_state <= "000";
        start_state_out <= "000";
        start_state<= 0;
        count_disp <= 0;
    end if;
else
    start_state_out <= "000";
    count_disp <= 0;
    text_state <= "000";
end if;
end if;
end if;
end process;
end Behavioral;

```


Cocktail.xdc

##Sch name = JC1

set_property PACKAGE_PIN K17 [get_ports {lcd_db[3]]

set_property IOSTANDARD LVCMOS33 [get_ports {lcd_db[3]]

##Sch name = JC2

set_property PACKAGE_PIN M18 [get_ports {lcd_db[2]]

set_property IOSTANDARD LVCMOS33 [get_ports {lcd_db[2]]

##Sch name = JC3

set_property PACKAGE_PIN N17 [get_ports {lcd_db[1]]

set_property IOSTANDARD LVCMOS33 [get_ports {lcd_db[1]]

##Sch name = JC4

set_property PACKAGE_PIN P18 [get_ports {lcd_db[0]]

set_property IOSTANDARD LVCMOS33 [get_ports {lcd_db[0]]

##Sch name = JC7

set_property PACKAGE_PIN L17 [get_ports {lcd_db[7]]

set_property IOSTANDARD LVCMOS33 [get_ports {lcd_db[7]]

##Sch name = JC8

set_property PACKAGE_PIN M19 [get_ports {lcd_db[6]]

set_property IOSTANDARD LVCMOS33 [get_ports {lcd_db[6]]

##Sch name = JC9

set_property PACKAGE_PIN P17 [get_ports {lcd_db[5]]

set_property IOSTANDARD LVCMOS33 [get_ports {lcd_db[5]]

##Sch name = JC10

set_property PACKAGE_PIN R18 [get_ports {lcd_db[4]]

set_property IOSTANDARD LVCMOS33 [get_ports {lcd_db[4]]

set_property PACKAGE_PIN W5 [get_ports clk]

set_property IOSTANDARD LVCMOS33 [get_ports clk]

##Pmod Header JB

##Sch name = JB7

set_property PACKAGE_PIN A15 [get_ports {lcd_e}]

```
        set_property IOSTANDARD LVCMOS33 [get_ports {lcd_e}]  
##Sch name = JB8  
set_property PACKAGE_PIN A17 [get_ports {lcd_rw}]  
        set_property IOSTANDARD LVCMOS33 [get_ports {lcd_rw}]  
##Sch name = JB9  
set_property PACKAGE_PIN C15 [get_ports {lcd_rs}]  
        set_property IOSTANDARD LVCMOS33 [get_ports {lcd_rs}]
```

```
set_property PACKAGE_PIN A14 [get_ports buton_right]  
set_property IOSTANDARD LVCMOS33 [get_ports buton_right]  
set_property PACKAGE_PIN A16 [get_ports buton_select]  
set_property IOSTANDARD LVCMOS33 [get_ports buton_select]  
set_property PACKAGE_PIN B15 [get_ports buton_left]  
set_property IOSTANDARD LVCMOS33 [get_ports buton_left]
```

```
##Sch name = JB4  
set_property PACKAGE_PIN B16 [get_ports {sensor}]  
        set_property IOSTANDARD LVCMOS33 [get_ports {sensor}]
```

```
set_property PACKAGE_PIN U16 [get_ports {pump_out_1}]  
set_property IOSTANDARD LVCMOS33 [get_ports {pump_out_1}]  
set_property PACKAGE_PIN E19 [get_ports {pump_out_2}]  
set_property IOSTANDARD LVCMOS33 [get_ports {pump_out_2}]  
set_property PACKAGE_PIN U19 [get_ports {pump_out_3}]  
set_property IOSTANDARD LVCMOS33 [get_ports {pump_out_3}]
```

##Pmod Header JA

##Sch name = JA1

set_property PACKAGE_PIN J1 [get_ports {p1_pin1}]

set_property IOSTANDARD LVCMOS33 [get_ports {p1_pin1}]

##Sch name = JA2

set_property PACKAGE_PIN L2 [get_ports {p1_pin2}]

set_property IOSTANDARD LVCMOS33 [get_ports {p1_pin2}]

##Sch name = JA3

set_property PACKAGE_PIN J2 [get_ports {p2_pin1}]

set_property IOSTANDARD LVCMOS33 [get_ports {p2_pin1}]

##Sch name = JA4

set_property PACKAGE_PIN G2 [get_ports {p2_pin2}]

set_property IOSTANDARD LVCMOS33 [get_ports {p2_pin2}]

##Sch name = JA7

set_property PACKAGE_PIN H1 [get_ports {p3_pin1}]

set_property IOSTANDARD LVCMOS33 [get_ports {p3_pin1}]

##Sch name = JA8

set_property PACKAGE_PIN K2 [get_ports {p3_pin2}]

set_property IOSTANDARD LVCMOS33 [get_ports {p3_pin2}]